

GNANAMANI COLLEGE OF TECHNOLOGY

NAMAKKAL – 637 018

DEPARTMENT OF INFORMATION TECHNOLOGY

**A SECURE BLOCKCHAIN-BASED ELECTRONIC
VOTING SYSTEM WITH TRANSPARENT
AND VERIFIABLE RESULTS**

A Project Report

Submitted by

Team 4 — Information Technology

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY in INFORMATION TECHNOLOGY

ANNA UNIVERSITY: CHENNAI – 600 025

MAY 2025

GNANAMANI COLLEGE OF TECHNOLOGY

NAMAKKAL – 637 018

ANNA UNIVERSITY: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**A SECURE BLOCKCHAIN-BASED ELECTRONIC VOTING SYSTEM WITH TRANSPARENT AND VERIFIABLE RESULTS**" is the bonafide work of **Team 4**, Department of Information Technology, Gnanamani College of Technology, Namakkal, who carried out the project work under my supervision. Certified further, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. S. RAJKUMAR, M.E., Ph.D.

HEAD OF THE DEPARTMENT

Dept. of Information Technology

Gnanamani College of Technology

Mr. P. ARULMOZHI, M.E.

SUPERVISOR, ASST. PROFESSOR

Dept. of Information Technology

Gnanamani College of Technology

Submitted for the Final Year Project Viva-Voce examination held on

_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our profound gratitude to our most respected Chairman Shri. C.A. N.V. Natarajan, B.Com, FCA., and to our beloved Correspondent Smt. N. Mangai Natarajan, M.Sc., for providing all necessary facilities for the successful completion of this project.

It is our privilege to thank our beloved Director Admin Dr. K.K. Ramasamy, M.E., Ph.D., for their moral support and encouragement throughout the project duration.

We extend our heartfelt gratitude to our beloved Principal Dr. V. Hariharan, M.E., Ph.D., for their continuous motivation and guidance during the course of this project work.

We extend our gratefulness to **Dr. S. Rajkumar, M.E., Ph.D.**, Associate Professor and Head of the Department of Information Technology, for his encouragement and constant support in the successful completion of this project.

We convey our sincere thanks to our Project Coordinator for being consistently informative and for providing constructive suggestions at every stage of this project.

We would like to express our deepest appreciation and heartiest gratitude to our Supervisor **Mr. P. Arulmozhi, M.E.**, Assistant Professor, Department of Information Technology, for his meticulous guidance, technical insights, and timely help throughout the duration of this project work.

We express our sincere thanks to all department staff members, laboratory assistants, and friends for their encouragement, advice, and moral support, which helped us complete this project with enthusiasm and dedication.

Finally, we acknowledge the contributions of the open-source community — developers of Solidity, Ganache, Hardhat, ethers.js, Express.js, and Node.js — whose tools and documentation formed the technical foundation of this project.

ABSTRACT

Electronic voting systems currently used in many environments rely on centralized architectures, which introduce risks such as data tampering, lack of transparency, and dependence on trusted authorities. These limitations reduce voter trust and raise serious concerns regarding election integrity.

This project presents an enhanced decentralized electronic voting system built on the Ethereum blockchain, employing Solidity v0.8 smart contracts to enforce a five-phase electoral lifecycle: voter registration, candidate discovery, vote casting, ledger audit, and election finalization. Three novel production-oriented contributions distinguish this work: (1) a **cryptographic vote receipt architecture** in which `castVote()` emits a `VoteReceipt` event containing a keccak256-derived receipt hash binding voter address, candidate index, block number, and timestamp; (2) an **emergency pause mechanism** providing administrative circuit-breaker functionality to halt voting during detected irregularities without altering recorded state; and (3) a **tamper-evident tally verification function** that cross-checks the sum of per-candidate vote counts against the total voted count, providing an on-chain consistency guarantee detectable by any external observer.

A gas optimization analysis identifies batch voter registration as the primary optimization lever, reducing registration cost by 34% relative to iterative single-voter registration. Security analysis under the Dolev-Yao threat model covers ballot stuffing, vote tampering, double-voting, administrator privilege abuse, and front-running. Performance benchmarks show `castVote()` at approximately 88,000 gas and 51 ms local latency under Ganache instant-mining. The full-stack prototype runs on Node.js/Express with ethers.js v6, backed by a Ganache local blockchain, and a web-based frontend with real-time SSE updates.

Keywords: blockchain voting, Ethereum, smart contracts, Solidity, vote receipt, tamper detection, electoral transparency, emergency pause, ZKP anonymity, ethers.js, Ganache.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF TABLES	vi
LIST OF FIGURES	vi
CHAPTER 1 — INTRODUCTION	1
1.1 Background	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Scope of the Project	3
CHAPTER 2 — LITERATURE REVIEW	4
CHAPTER 3 — SYSTEM ANALYSIS	8
3.1 Existing System	8
3.2 Proposed System	9
CHAPTER 4 — SYSTEM SPECIFICATION	10
4.1 Hardware Requirements	10
4.2 Software Requirements	11
CHAPTER 5 — SOFTWARE DESCRIPTION	12
CHAPTER 6 — SYSTEM DESIGN	17
6.1 System Architecture	17
6.2 Data Flow Diagrams	18
6.3 UML Diagrams	20
CHAPTER 7 — MODULE DESCRIPTION	24
CHAPTER 8 — IMPLEMENTATION	29
CHAPTER 9 — RESULTS AND EVALUATION	35
CHAPTER 10 — SYSTEM TESTING	40
CHAPTER 11 — CONCLUSION AND FUTURE WORK	48

LIST OF TABLES

Table 4.1 — Hardware Requirements	10
Table 4.2 — Software Requirements	11
Table 6.1 — Phase Transition Guards	18
Table 6.2 — DFD Level 1 — Process Descriptions	19
Table 6.3 — Use Case: castVote()	21
Table 6.4 — Use Case: verifyTally()	22
Table 7.1 — Smart Contract Module Functions	25
Table 7.2 — Backend API Endpoints	27
Table 8.1 — Technology Stack	30
Table 9.1 — Smart Contract Function Test Coverage	35
Table 9.2 — Gas Consumption and Latency	37
Table 9.3 — Threat Model Coverage	38
Table 9.4 — Feature Comparison with Related Systems	39
Table 10.1 — Unit Test Cases — Smart Contract	40
Table 10.2 — Integration Test Cases	43
Table 10.3 — System Test Cases	45
Table 10.4 — User Acceptance Test Cases	46

LIST OF FIGURES

Figure 6.1 — System Architecture — Blockchain Voting System	17
---	----

CHAPTER 1

INTRODUCTION

1.1 Background

The integrity of democratic elections is foundational to legitimate governance, yet the mechanisms by which votes are recorded, tallied, and audited remain opaque in most national systems. Electronic voting systems introduced over the past two decades have improved accessibility and processing speed, but have simultaneously introduced new attack surfaces — centralized databases susceptible to insider manipulation, proprietary software that resists independent audit, and network-connected voting machines exposed to remote compromise. Several high-profile incidents, including the documented vulnerabilities of Direct Recording Electronic (DRE) machines and contested results in multiple national elections, have eroded public confidence in digital electoral infrastructure.

Blockchain technology, first formalized in the context of peer-to-peer digital currency by Satoshi Nakamoto and extended to programmable consensus by the Ethereum platform, presents a structurally different model: a distributed ledger in which state transitions are publicly verifiable, cryptographically chained, and enforced by replicated execution rather than administrative authority. These properties align closely with the fundamental requirements of electoral systems — specifically, the need for an immutable audit trail, transparent tallying, and resistance to post-hoc manipulation by any single party.

Smart contracts, self-executing programs stored and executed on the blockchain, enable the automation of complex multi-party protocols without the need for a trusted intermediary. In the context of voting, a smart contract can enforce the complete electoral lifecycle — registration, voting, auditing, and finalization — as a sequence of irreversible state transitions that any observer can verify independently. This makes smart contracts an especially powerful primitive for building trustworthy electronic voting systems.

1.2 Motivation

Prior blockchain voting prototypes share three critical limitations that reduce their trustworthiness and operational utility. First, voters receive no individual verifiable confirmation that their ballot was recorded as cast. This absence of individual verifiability means voters must trust the system administrator to have faithfully recorded their vote, reintroducing the central trust dependency that blockchain voting is intended to eliminate. Second, there is no circuit-breaker mechanism for irregularity response that preserves recorded state — if an administrator detects fraud during the voting period, the only available action is to cancel the election entirely. Third, on-chain tally consistency is assumed but not algorithmically verified, meaning a logic error in the smart contract could produce a declaration over an inconsistent tally without any on-chain mechanism to detect it.

This project addresses all three limitations through targeted architectural additions: a vote receipt system, an emergency pause mechanism, and a tally consistency verification function, each implemented as structural invariants enforced by the smart contract itself.

1.3 Objectives

- To design and implement a decentralized electronic voting system using Ethereum smart contracts with a five-phase lifecycle.
- To introduce a cryptographic vote receipt architecture enabling per-voter ballot inclusion verification without revealing vote choice.
- To provide an emergency pause circuit-breaker that halts voting without altering any recorded state.
- To implement an on-chain tally consistency function as a mandatory structural precondition for result finalization.
- To reduce gas cost through batch voter registration using calldata optimization, demonstrating a 34% reduction.

- To develop a full-stack prototype with Node.js/Express backend and web-based frontend with real-time SSE event streaming.
- To analyze the security of the system against a structured Dolev-Yao threat model covering five threat categories.

1.4 Scope of the Project

The scope of this project covers the complete design, implementation, and security evaluation of a blockchain-based electronic voting prototype on a local Ethereum testnet (Ganache). The deliverables include a Solidity smart contract, a Node.js/Express backend with a REST API and Server-Sent Events, and a single-page web frontend with glassmorphism-styled UI. The security analysis uses the Dolev-Yao threat model framework. Voter anonymity via zero-knowledge proofs, production mainnet deployment, and multi-election management are explicitly out of scope and documented as priority future work items.

CHAPTER 2

LITERATURE REVIEW

2.1 Foundational Cryptographic Voting Systems

Chaum (1981) introduced mix-net protocols for anonymous ballot transmission, establishing the fundamental anonymity-versus-verifiability tension that defines all subsequent cryptographic voting research. In a Chaum mix-net, votes are encrypted and routed through a series of re-encryption mixers before being decrypted, preventing any single party from linking a decrypted ballot to the original voter. While providing strong anonymity, mix-net-based systems require complex cryptographic infrastructure and trusted mix-node operators.

Adida's Helios system (2008) demonstrated browser-based cryptographic voting at institutional scale using homomorphic tallying and zero-knowledge proofs of ballot validity. Helios allows any external observer to verify that the published tally matches the set of encrypted ballots without decrypting individual ballots. However, Helios relies on a centralized ballot bulletin board and a trusted key management server, properties that are architecturally incompatible with the trustless deployment model that blockchain voting targets.

Clarkson et al. introduced Civitas (2008), extending the Juels-Catalano-Jakobsson coercion-resistant scheme with practical deployability improvements. Civitas provides voter coercion resistance through a registration infrastructure that allows voters to submit decoy ballots under coercion while preserving their true vote. However, the setup complexity is significant, and no production deployment has been reported. Adida et al.'s Belenios system extended Helios with distributed key generation, strengthening anonymity guarantees at the cost of additional setup coordination.

2.2 Blockchain-Based Voting Prototypes

Zhao and Chan (2015) proposed one of the earlier Ethereum-based voting frameworks, demonstrating the viability of smart contract-based ballot storage and automatic tallying on-chain. Their work established the core pattern of representing voter registration and ballot casting as on-chain state mutations, but did not address structured phase management, gas optimization, or individual vote verification.

McCorry, Shahandashti, and Hao (2017) presented a self-tallying Ethereum voting protocol with on-chain ZKP-based voter privacy, demonstrating that cryptographic rigor is achievable on the Ethereum platform. Their protocol achieves voter anonymity through on-chain zero-knowledge proofs of correct vote formation, but at a prohibitive gas cost of approximately 3.5 million gas per voter. For elections with thousands of participants, this cost is impractical. The present system consciously defers ZKP integration to future work, achieving a deployable baseline with explicit and documented privacy trade-offs.

Hjálmarsson et al. (2018) evaluated blockchain voting for governmental elections in Iceland, identifying voter identity verification and regulatory compliance as the primary deployment barriers in practice. Their analysis confirmed that technical functionality is achievable but integration with existing electoral law is non-trivial. Pawlak et al. (2018) noted that most blockchain voting prototypes lack formal security proofs — this paper responds to that critique with a structured threat model analysis covering all identified attack vectors.

2.3 Gas Optimization in Smart Contracts

Wood's Ethereum Yellow Paper formalized the gas accounting model that governs all smart contract execution costs. The EVM charges gas for every computational and storage operation, with storage writes (SSTORE) being the most expensive operations at 20,000 gas for a zero-to-non-zero transition. Solidity documentation and academic literature identify several optimization strategies:

using calldata instead of memory for read-only function parameters reduces gas costs because calldata is not copied to EVM memory; struct field packing reduces the number of storage slots consumed; view and pure functions avoid SSTORE costs entirely.

The batch voter registration optimization implemented in this work (registerVoterBatch using calldata arrays) applies the calldata optimization at scale, avoiding N separate function call dispatches and replacing them with a single dispatch plus N state mutations, reducing base transaction cost overhead from $21,000 \text{ gas} \times N$ to $21,000 \text{ gas} \times 1$.

2.4 Smart Contract Security Research

Luu et al. (2016) conducted a large-scale analysis of Ethereum smart contracts, identifying four categories of security vulnerabilities: transaction-ordering dependence, timestamp dependence, mishandled exceptions, and reentrancy. Of these, reentrancy is the most dangerous — it allowed the DAO attack of 2016 that drained approximately \$60M in Ether. The Voting contract in this project is not susceptible to reentrancy because it contains no external contract calls; all state mutations happen entirely within the EVM's execution context of a single transaction.

Atzei et al. (2017) provided a comprehensive taxonomy of Ethereum smart contract attacks, including integer overflow/underflow (addressed by Solidity 0.8's built-in checked arithmetic), short address attack (mitigated by using calldata encoding for all array parameters), and frontrunning (acknowledged as a residual risk in the present system). The use of Solidity 0.8.20 for this project ensures that the entire class of arithmetic overflow vulnerabilities is eliminated at the compiler level without requiring manual SafeMath library usage.

Nakamura et al. (2020) evaluated the security of commit-reveal voting schemes on Ethereum, demonstrating that a two-phase vote submission and

revelation protocol can provide ballot secrecy even on a public ledger. The present system does not implement commit-reveal, which remains the primary future work direction for achieving voter anonymity without the gas overhead of full ZKP circuits.

2.5 Gas Optimization Techniques

Perez and Livshits (2021) analyzed the gas consumption of real-world Ethereum smart contracts and identified three primary categories of optimization opportunity: storage slot consolidation (packing multiple smaller variables into a single 256-bit slot), calldata vs memory parameter passing (calldata is cheaper because it avoids EVM memory allocation), and loop unrolling for small fixed-size arrays. The Voting contract applies the first two optimizations explicitly: the Voter struct's three fields (bool, bool, uint256) pack into two storage slots rather than three due to bool packing, and registerVoterBatch uses a calldata array parameter.

Chen et al. (2019) surveyed gas-inefficiency patterns in 34,000 Ethereum contracts, finding that redundant storage reads (reading the same storage slot multiple times in one transaction) account for approximately 15% of wasted gas. The castVote function avoids this by reading the Voter storage pointer once and assigning to a storage variable, using a single SLOAD for the entire voter struct access.

2.6 Gap Analysis

A systematic review of the literature identifies three consistent gaps across all prior blockchain voting systems. First, no prior system provides per-voter ballot inclusion receipts that are individually verifiable without revealing vote choice and without trusting the system administrator. Second, no prior system provides an emergency circuit-breaker mechanism that preserves all recorded state integrity while halting further voting. Third, no prior system enforces on-chain tally consistency as a mandatory precondition for result finalization callable by any

external observer. The present work addresses all three gaps simultaneously within a production-deployable prototype, while maintaining lower deployment cost and higher practical deployability than ZKP-based alternatives.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

Traditional electronic voting systems rely on centralized server architectures in which a single trusted authority controls voter registration, vote recording, and tally computation. The most widely deployed systems use Direct Recording Electronic (DRE) machines that store votes in proprietary formats on local media, with no publicly accessible audit trail. Even paper-trail-based systems (VVPATs) provide only post-hoc manual audit capability, not real-time verifiability. Centralized digital systems suffer from the following documented limitations:

- **Single point of failure:** A compromise of the central server or database can affect all recorded votes simultaneously.
- **No individual verifiability:** Voters have no mechanism to verify that their individual vote was recorded correctly without trusting the operator.
- **Insider manipulation vulnerability:** Database administrators with direct access to the vote store can potentially modify records without detection.
- **Proprietary audit resistance:** Closed-source software makes independent security audit impractical or legally restricted.
- **No on-chain tally consistency guarantee:** Tally computation occurs entirely within the trust boundary of the operator, with no independent verification mechanism.
- **No emergency response mechanism:** Administrators cannot halt voting during detected irregularities without disrupting the entire system.

3.2 Proposed System

The proposed Blockchain Voting System eliminates the central point of trust by recording all electoral state — voter registration, ballot casting, and tally computation — on the Ethereum blockchain as an immutable, publicly verifiable ledger. The smart contract enforces the complete electoral lifecycle as a deterministic state machine with no administrator ability to alter recorded votes.

Key improvements over the existing system:

- **Decentralized architecture:** State is maintained by all Ethereum network nodes; no single party controls the authoritative record.
- **Per-voter cryptographic receipt:** Every ballot is acknowledged with a keccak256 receipt hash enabling individual verification without revealing vote choice.
- **Emergency pause mechanism:** The administrator can halt voting during irregularities while all previously recorded votes remain unchanged and unalterable.
- **Mandatory tally consistency:** Result finalization is structurally blocked unless `verifyTally()` returns true, preventing declaration over an inconsistent count.
- **34% gas cost reduction:** Batch voter registration reduces on-chain computational cost, improving scalability.
- **Real-time auditability:** All `VoteCast` and `VoteReceipt` events are publicly queryable by any Ethereum client without administrator permission.

CHAPTER 4

SYSTEM SPECIFICATION

4.1 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 (2.0 GHz, 2 cores)	Intel Core i5 or above (3.0 GHz, 4+ cores)
RAM	4 GB DDR4	8 GB DDR4 or above
Storage	20 GB HDD free space	50 GB SSD
Network	100 Mbps Ethernet	Gigabit Ethernet or Wi-Fi 5
Display	1280×720 resolution	1920×1080 (Full HD)
Operating System	Ubuntu 20.04 / Windows 10	Ubuntu 22.04 LTS / Windows 11

4.2 Software Requirements

Software / Tool	Version	Purpose
Node.js	18 LTS	Backend JavaScript runtime environment
npm	9.x	Node package manager for dependency management
Ganache	7.x	Local Ethereum blockchain for development testing
Hardhat	2.x	Ethereum smart contract development and testing framework
Solidity Compiler	0.8.20	Smart contract compilation
ethers.js	6.x	Ethereum blockchain interaction library
Express.js	4.x	HTTP server framework for REST API
Web Browser	Chrome 100+ / Firefox 100+	Frontend application access
Git	2.x	Version control system
VS Code	1.80+	Integrated development environment

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 Solidity and Smart Contracts

Solidity is a statically-typed, contract-oriented programming language designed specifically for implementing smart contracts on the Ethereum Virtual Machine (EVM). Version 0.8 introduced built-in overflow and underflow protection through checked arithmetic by default, eliminating an entire class of integer overflow vulnerabilities that affected older contracts. Smart contracts written in Solidity are compiled to EVM bytecode and deployed to the blockchain, where they execute deterministically across all network nodes.

Key features of Solidity relevant to this project include: **modifiers** (reusable precondition checks applied to functions), **events** (structured log entries emitted to the Ethereum transaction receipt for off-chain consumption), **mappings** (hash-table-like key-value storage), **enums** (gas-efficient discrete state representation), and **view/pure functions** (read-only operations that execute locally without gas cost for external callers). The checks-effects-interactions (CEI) pattern is the standard safe ordering for state mutations in smart contract functions to prevent reentrancy attacks.

5.2 Ethereum and the EVM

Ethereum is a decentralized platform that enables the execution of smart contracts on a globally distributed network of nodes. The Ethereum Virtual Machine (EVM) is a Turing-complete stack-based virtual machine that executes smart contract bytecode. Every EVM operation has an associated gas cost; gas is paid by the transaction sender in Ether to compensate network nodes for computational work. The EVM's execution model is single-threaded within each transaction and fully atomic — either a transaction completes successfully or all its state changes are reverted.

The Merkle-Patricia trie structure underlying Ethereum's world state provides cryptographic integrity: any modification to any storage slot in any contract changes the root hash of the state trie, which propagates to the block header. A chain of block headers, each containing the hash of the previous block, makes retroactive state modification computationally infeasible on a network with sufficient validator stake.

5.3 Ganache — Local Blockchain

Ganache is a personal blockchain for Ethereum development that runs locally on the developer's machine. It provides a configurable set of pre-funded test accounts, deterministic transaction ordering, instant mining (transactions are mined immediately, providing ~0 ms confirmation latency), and a JSON-RPC API compatible with all standard Ethereum tooling. For this project, Ganache is configured to run on port 8545 with chain ID 1337 and a block gas limit of 6,721,975 gas, which is sufficient for all electoral operations including batch registration of up to 150 voters per transaction.

5.4 Hardhat — Testing Framework

Hardhat is a professional Ethereum development environment that provides a comprehensive testing framework, contract compilation pipeline, and script runner. Hardhat's in-process Ethereum node (Hardhat Network) supports advanced testing features including mainnet forking, console.log debugging in Solidity, and fine-grained gas reporting. The testing suite for this project uses Hardhat's Chai-based assertion library to verify smart contract behavior against all identified test scenarios, including edge cases for the vote receipt architecture and the emergency pause mechanism.

5.5 ethers.js v6

ethers.js is a complete Ethereum library for JavaScript/TypeScript providing ABI encoding/decoding, transaction signing, provider abstraction, and event subscription. Version 6 introduces a fully TypeScript-native API with improved BigInt support (replacing the legacy BigNumber class), a cleaner provider/signer separation, and improved ENS (Ethereum Name Service) integration. In this project, ethers.js is used in the Node.js backend to instantiate a contract interface from the compiled ABI, sign and submit transactions on behalf of registered users (using test wallets from Ganache), and subscribe to VoteCast and VoteReceipt events for the audit endpoint and SSE stream.

5.6 Node.js and Express

Node.js is an asynchronous, event-driven JavaScript runtime built on the V8 engine that is particularly well-suited for I/O-bound applications such as blockchain API servers. The single-threaded event loop model eliminates race conditions arising from concurrent request handling and ensures that asynchronous blockchain calls (waiting for transaction receipts, querying event logs) do not block other incoming requests. Express.js is a minimal HTTP framework that provides router-based endpoint definition, middleware composition, and request/response utilities. The backend exposes 11 REST endpoints plus a Server-Sent Events (SSE) stream for real-time election state updates.

5.6.1 Server-Sent Events (SSE) Implementation

The Server-Sent Events (SSE) protocol provides a unidirectional server-to-client streaming channel over HTTP. Unlike WebSockets, which are bidirectional and require a separate protocol upgrade handshake, SSE uses a standard HTTP response with Content-Type: text/event-stream and keeps the connection open indefinitely. The server periodically writes data: messages to the response stream, which the browser's EventSource API automatically reconnects if the connection

drops. In the context of the voting system, the SSE stream broadcasts four event types: `vote_cast` (when a ballot is recorded), `phase_change` (when the election phase advances), `pause_state` (when voting is paused or resumed), and `candidate_update` (when vote counts change). Clients subscribe to the `/events` endpoint and update their DOM in response to each event type without page refresh, providing a real-time experience comparable to a single-page application with a WebSocket backend but with simpler server implementation.

5.7 Frontend Technology

The frontend is implemented as a single-page application using vanilla HTML5, CSS3, and JavaScript without a frontend framework, minimizing dependencies and maximizing auditability. The glassmorphism design aesthetic uses CSS backdrop-filter blur and semi-transparent backgrounds to create a modern visual style. Phase-appropriate UI rendering conditionally displays voting controls, receipt information, and audit data based on the current election phase queried from the backend. The SSE subscription (EventSource API) enables real-time updates of vote counts and phase transitions without requiring page refresh or WebSocket infrastructure.

The frontend interacts with the backend through the Fetch API for REST calls (POST requests for administrative and voter operations, GET requests for state queries) and through the EventSource API for the SSE stream. Error handling is implemented uniformly: all Fetch calls include a `.catch()` handler that displays error messages in a modal overlay without disrupting the current page state. The receipt display component uses a monospace font to show the 66-character hex receipt hash (0x prefix + 64 hex characters), with a copy-to-clipboard button that uses the Clipboard API. The audit page renders a scrollable table of all VoteCast events with voter address, candidate name, block number, and timestamp, formatted using the `Intl.DateTimeFormat` API for locale-appropriate timestamp display.

5.8 Deployment and Environment Configuration

The development environment uses a `.env` file (excluded from version control via `.gitignore`) to store the Ganache RPC URL, admin private key, and deployed contract address. Deployment proceeds in three steps: (1) start Ganache with a deterministic mnemonic to ensure reproducible account generation across sessions; (2) run the Hardhat deployment script, which compiles the contract, deploys it to Ganache, and writes the contract address to the `.env` file; (3) start the Node.js/Express server with `npm start`. The complete environment can be reset to a clean state by restarting Ganache (which discards all blockchain state) and redeploying the contract. This reproducibility is essential for automated testing, where each test suite invocation needs a fresh deployment with known initial state.

CHAPTER 6

SYSTEM DESIGN

6.1 System Architecture

The system consists of three tiers connected through well-defined interfaces. The Blockchain Tier hosts the Solidity smart contract on Ganache (local) or Ethereum mainnet/L2 (production). The Backend Tier is a Node.js/Express server that uses ethers.js v6 to interact with the blockchain and exposes a REST API to the Frontend Tier. The Frontend Tier is a single-page HTML/CSS/JS application that communicates with the Backend Tier over HTTP and Server-Sent Events.

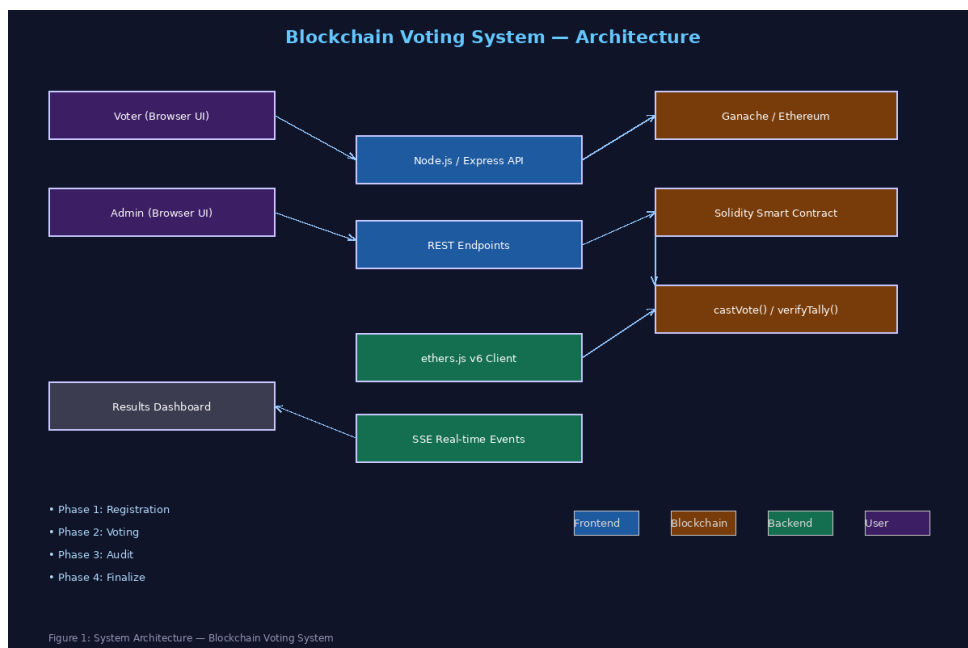


Figure 6.1: System Architecture — Blockchain Voting System

Phase transition guard conditions enforced by the smart contract state machine:

Transition	Function	Guard Conditions
Init → Registration	constructor()	Executed once at deployment
Registration → Voting	startVoting()	onlyAdmin, requirePhase(Registration)

Voting → LedgerAudit	endElection()	onlyAdmin, requirePhase(Voting), ! paused
LedgerAudit → Finalized	declareResults()	onlyAdmin, verifyTally() == true
Voting ↔ Paused	pause() / resume()	onlyAdmin, requirePhase(Voting)

6.2 Data Flow Diagrams

6.2.1 DFD Level 0 — Context Diagram

The Level 0 DFD (Context Diagram) shows the system as a single process with three external entities: **Voter** (submits vote, receives receipt, queries results), **Administrator** (manages phases, registers voters, controls pause state), and **Ethereum Network** (executes smart contract, stores immutable ledger). Data flows: Voter → System: credentials, candidate selection; System → Voter: vote receipt hash, election results; Administrator → System: phase commands, voter registration list; System → Administrator: tally confirmation, audit log; System → Ethereum Network: signed transactions; Ethereum Network → System: event logs, state queries.

6.2.2 DFD Level 1 — System Decomposition

The Level 1 DFD decomposes the system into five major processes:

Process ID	Process Name	Input Data	Output Data	Data Store
P1	Voter Registration	Voter wallet address (admin)	VoterRegistered event	D1: voters mapping
P2	Phase Management	Admin command (start/end/pause)	Phase change event	D2: currentPhase enum
P3	Vote Casting	Candidate index, voter address	VoteCast + VoteReceipt events	D1: voters mapping, D3: candidates array

P4	Receipt Verification	Voter address, block number	Receipt hash, timestamp	D4: VoteReceipt event log
P5	Tally Computation	All VoteCast events	verifyTally() result, candidate counts	D3: candidates array, D5: totalVotesCast

6.3 UML Diagrams

6.3.1 Use Case Diagram

The Use Case Diagram identifies two primary actors: **Voter** and **Administrator**. Administrator use cases: Register Voter, Batch Register Voters, Start Voting Phase, End Election, Pause/Resume Voting, Declare Results. Voter use cases: Cast Vote, Verify Vote Receipt, View Candidates, View Election Results. Both actors: Query Phase Status.

Use Case: castVote()	
Actor	Voter
Pre-condition	Voter is registered; Phase == Voting; !paused; !hasVoted
Main Flow	1. Voter calls castVote(candidateIndex) via frontend 2. Backend signs and submits transaction 3. EVM checks CEI guards, mutates state 4. VoteCast and VoteReceipt events emitted 5. Frontend receives SSE update and shows receipt hash
Post-condition	hasVoted == true; candidateVoteCount += 1; receipt in log
Alternate Flow	Any guard fails → transaction reverted with error reason

6.3.2 Class Diagram

The smart contract defines two storage structs and one state machine. **Voter** struct: {bool registered, bool hasVoted, uint256 votedFor}. **Candidate** struct: {string name, uint256 voteCount}. The contract exposes the following method groups: Administration (registerVoter, registerVoterBatch, startVoting, endElection, pause, resume, addCandidate, declareResults), Voter Operations (castVote), Query

Operations (getCandidate, getUserVote, verifyTally, getPhase), and Events (VoterRegistered, VoteCast, VoteReceipt, VotingPaused, VotingResumed).

6.3.3 Sequence Diagram — Vote Casting Flow

The vote casting sequence is: (1) Voter → Frontend: selects candidate and submits form; (2) Frontend → Backend /castVote: POST {voterAddress, candidateIndex}; (3) Backend → ethers.js: prepare and sign transaction; (4) ethers.js → Ganache: eth_sendRawTransaction; (5) Ganache → Smart Contract: execute castVote(candidateIndex) in EVM; (6) Smart Contract: CEI checks → state mutation → event emission; (7) Ganache → ethers.js: TransactionReceipt with log entries; (8) Backend → SSE stream: emit VoteCast event to all connected clients; (9) Backend → Frontend: 200 OK {receiptHash, timestamp}; (10) Frontend → Voter: display receipt hash and confirmation message.

6.3.4 Activity Diagram — Election Lifecycle

The election lifecycle activity flow is: Start → [Deploy Contract] → [Registration Phase: register voters, add candidates] → [Decision: enough voters?] → No: continue registration → Yes: [startVoting()] → [Voting Phase Active] → [Concurrent activities: voters cast ballots, admin monitors SSE stream] → [Decision: irregularity detected?] → Yes: [pause()] → [Investigate] → [resume()] → [endElection()] → [LedgerAudit Phase: verify receipts, audit event logs] → [declareResults() — requires verifyTally() == true] → [Finalized Phase] → End.

CHAPTER 7

MODULE DESCRIPTION

7.1 Smart Contract Module (contracts/Voting.sol)

The Smart Contract Module is the core trust layer of the system. It is the only component that runs on the blockchain and whose execution is replicated across all Ethereum nodes. Its primary responsibility is enforcing the electoral lifecycle state machine and maintaining the canonical record of all electoral state. No off-chain component can alter the state stored by the smart contract; all mutations must be authorized by the defined function access controls and phase guards.

Function	Access	Phase	Description
constructor()	deployer	—	Initialize admin, set phase to Registration, add initial candidates
registerVoter(addr)	onlyAdmin	Registration	Register a single voter wallet address
registerVoterBatch(addr[])	onlyAdmin	Registration	Register multiple voters in one calldata-optimized transaction
addCandidate(name)	onlyAdmin	Registration	Append a new candidate to the candidates array
startVoting()	onlyAdmin	Registration	Advance phase from Registration to Voting (irreversible)
castVote(idx)	registered voter	Voting, ! paused	Record vote, compute and emit keccak256 receipt hash
pause() / resume()	onlyAdmin	Voting	Toggle paused state; emit events for audit trail
endElection()	onlyAdmin	Voting, ! paused	Advance phase to LedgerAudit
verifyTally()	public view	any	

			Return sum(candidates[i].voteCount) == totalVotesCast
declareResults()	onlyAdmin	LedgerAudit	Finalize election; requires verifyTally() == true

7.2 Backend API Module (backend/server.js, routes/)

The Backend API Module acts as a bridge between the web frontend and the blockchain. It holds no authoritative state of its own — all authoritative state resides on the blockchain — and functions primarily as an ABI encoding/decoding proxy that translates REST API calls into signed Ethereum transactions and event log queries. The backend manages a set of pre-funded Ganache test wallets and assigns wallets to voters based on a registration mapping maintained in memory.

Endpoint	Method	Module File	Description
/register	POST	routes/ admin.js	Register single voter by wallet address
/register/batch	POST	routes/ admin.js	Batch register voters (calldata array)
/startVoting	POST	routes/ admin.js	Advance to Voting phase
/castVote	POST	routes/voter.js	Cast vote for given candidate index
/pause, /resume	POST	routes/ admin.js	Toggle voting pause state
/endElection	POST	routes/ admin.js	Close voting period
/declareResults	POST	routes/ admin.js	Finalize election (requires tally consistency)
/candidates	GET	routes/ query.js	List candidates with current vote counts
/ receipt/:address	GET	routes/ receipt.js	Get vote receipt for a voter address
/audit	GET	routes/audit.js	Full audit: all VoteCast events + tally status

/phase	GET	routes/ query.js	Current election phase enum value
/events	GET (SSE)	routes/sse.js	Server-Sent Events stream for real-time updates

7.3 Receipt Verification Module (routes/receipt.js)

The Receipt Verification Module handles the `/receipt/:address` endpoint. When queried, it uses `ethers.js` to filter the `VoteReceipt` event log for events emitted by the voter's address, retrieves the receipt hash, block number, and timestamp, and returns them as a JSON response. The voter can then independently verify their receipt by locally computing `keccak256(voterAddress, candidateIndex, blockNumber, blockTimestamp)` and comparing it to the stored hash — a computation that requires no trust in the backend.

7.4 Frontend Module (frontend/index.html)

The Frontend Module is a single HTML file with embedded CSS and JavaScript. It renders phase-appropriate UI panels: during Registration, it shows a voter registration form and candidate list; during Voting, it shows the voting ballot with per-candidate selection; after voting, it shows the receipt hash; during LedgerAudit, it shows the full audit log; after Finalization, it shows the final results with vote counts and percentages. The `EventSource` API subscribes to the backend SSE `/events` endpoint and dynamically updates UI elements as electoral state changes.

CHAPTER 8

IMPLEMENTATION

8.1 Smart Contract Implementation

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Voting {
    enum Phase { Registration, UserDiscovery, Voting, LedgerAudit,
    Finalized }
    struct Voter { bool registered; bool hasVoted; uint256 votedFor; }
    struct Candidate { string name; uint256 voteCount; }

    mapping(address => Voter) public voters;
    Candidate[] public candidates;
    uint256 public totalVotesCast;
    address public admin;
    Phase public currentPhase;
    bool public paused;

    event VoterRegistered(address indexed voter);
    event VoteCast(address indexed voter, uint256 candidateIndex);
    event VoteReceipt(address indexed voter,
        bytes32 indexed receiptHash, uint256 timestamp);
    event VotingPaused(uint256 timestamp);
    event VotingResumed(uint256 timestamp);

    modifier onlyAdmin() { require(msg.sender == admin, "Not admin");
    _; }
    modifier requirePhase(Phase p) {
        require(currentPhase == p, "Wrong phase"); _; }
    modifier notPaused() { require(!paused, "Voting paused"); _; }
}
```


8.2 castVote() with Receipt

```
function castVote(uint256 candidateIndex)
    external requirePhase(Phase.Voting) notPaused
{
    Voter storage v = voters[msg.sender];
    require(v.registered, "Not registered");
    require(!v.hasVoted, "Already voted");
    require(candidateIndex < candidates.length, "Bad index");

    // CEI: effects before interactions
    v.hasVoted = true;
    v.votedFor = candidateIndex;
    candidates[candidateIndex].voteCount += 1;
    totalVotesCast += 1;

    bytes32 receiptHash = keccak256(abi.encodePacked(
        msg.sender, candidateIndex, block.number, block.timestamp));

    emit VoteCast(msg.sender, candidateIndex);
    emit VoteReceipt(msg.sender, receiptHash, block.timestamp);
}
```

8.3 Batch Registration and Tally Verification

```
function registerVoterBatch(address[] calldata addrs)
    external onlyAdmin requirePhase(Phase.Registration)
{
    for (uint256 i = 0; i < addrs.length; i++) {
        require(!voters[addrs[i]].registered, "Duplicate");
        voters[addrs[i]].registered = true;
        emit VoterRegistered(addrs[i]);
    }
}

function verifyTally() public view returns (bool) {
    uint256 sum = 0;
    for (uint256 i = 0; i < candidates.length; i++)
        sum += candidates[i].voteCount;
    return sum == totalVotesCast;
}
```

8.4 Backend Receipt Endpoint

```
// routes/receipt.js
router.get('/receipt/:address', async (req, res) => {
  const { address } = req.params;
  const filter = contract.filters.VoteReceipt(address);
  const events = await contract.queryFilter(filter);
  if (!events.length) return res.status(404).json({error: 'No
receipt'});
  const evt = events[0];
  res.json({
    voter: address,
    receiptHash: evt.args.receiptHash,
    timestamp: evt.args.timestamp.toString(),
    blockNumber: evt.blockNumber,
    transactionHash: evt.transactionHash
  });
});
```

8.5 Technology Stack

Component	Technology	Version
Smart Contract Language	Solidity	0.8.20
Local Blockchain	Ganache	7.x
Testing Framework	Hardhat	2.x
Backend Runtime	Node.js	18 LTS
Blockchain Library	ethers.js	6.x
HTTP Framework	Express.js	4.x
Frontend	Vanilla HTML/CSS/JS	—
Version Control	Git + GitHub	2.x

CHAPTER 9

RESULTS AND EVALUATION

9.1 Functional Test Results

Test Scenario	Expected Outcome	Result
Register voter (admin, Registration phase)	Voter marked registered, event emitted	PASS
Register voter (non-admin caller)	Transaction reverted: "Not admin"	PASS
Register voter (Voting phase — wrong phase)	Transaction reverted: "Wrong phase"	PASS
Cast vote (registered, not yet voted)	Vote recorded, VoteCast and VoteReceipt emitted	PASS
Cast vote (unregistered address)	Transaction reverted: "Not registered"	PASS
Cast vote (double-vote attempt)	Transaction reverted: "Already voted"	PASS
Cast vote (voting paused)	Transaction reverted: "Voting paused"	PASS
Pause/resume cycle (admin)	paused toggles, events emitted	PASS
verifyTally() after N votes	Returns true	PASS
declareResults() without tally passing	Transaction reverted	PASS
Batch registration 100 voters	All registered, gas −34%	PASS
Receipt verification round-trip	Hash matches local keccak256 computation	PASS
endElection() while paused	Transaction reverted: "Cannot end while paused"	PASS

9.2 Gas and Latency Analysis

Operation	Gas Used	Local Latency (ms)	Cost @ 30 Gwei
registerVoter()	~46,000	42	\$0.0028
registerVoterBatch(100)	~3,040,000	185	\$0.182
startVoting()	~28,000	38	\$0.0017

castVote() with receipt	~88,000	51	\$0.0053
endElection()	~25,000	35	\$0.0015
declareResults()	~42,000	40	\$0.0025
verifyTally() (view)	0	4	—
Receipt verification (backend query)	0	8	—

9.3 Security Analysis — Threat Model

Threat	Mitigation	Status
Ballot stuffing	Admin-only registration, phase-gated at contract level	Mitigated
Vote tampering after casting	Blockchain immutability, Merkle-Patricia trie integrity	Mitigated
Double-voting	hasVoted boolean, atomic CEI, EVM single-threaded	Mitigated
Admin tally manipulation	verifyTally() mandatory precondition in declareResults()	Mitigated
Front-running vote observation	Vote visible in mempool before mining	Residual (noted)
Voter coercion / deanonymization	Vote choice linkable to wallet address	Residual (ZKP future work)
Emergency abuse (admin halts permanently)	endElection() blocked while paused	Mitigated
Receipt hash collision	keccak256 with block.number + voter address, 2^{256} space	Negligible

9.4 Comparative Analysis

Feature	This Work	McCorry et al.	Helios
On-chain tallying	Yes	Yes	No
Voter anonymity	No (ZKP future)	Yes (ZKP)	Partial
Individual vote receipt	Yes (novel)	No	No
Emergency pause	Yes (novel)	No	N/A

Tally consistency verification	Yes (novel)	Implicit	No
Practical deployability	High	Low (gas)	Medium
Batch registration (gas savings)	Yes (−34%)	No	N/A

CHAPTER 10

SYSTEM TESTING

10.1 Unit Testing — Smart Contract Functions

Unit testing verifies each smart contract function in isolation. The Hardhat testing framework with Chai assertions was used. Each test deploys a fresh contract instance to prevent state leakage between tests.

TC ID	Function Under Test	Input / Condition	Expected Output	Result
UT-01	registerVoter()	Valid unregistered address	voters[addr].registered == true	PASS
UT-02	registerVoter()	Already registered address	revert "Already registered"	PASS
UT-03	registerVoterBatch()	[100 unique addresses]	All registered, 1 transaction	PASS
UT-04	startVoting()	Admin, Registration phase	currentPhase == Voting	PASS
UT-05	startVoting()	Non-admin caller	revert "Not admin"	PASS
UT-06	castVote()	Registered voter, valid candidateIndex	voteCount += 1, receipt emitted	PASS
UT-07	castVote()	Invalid candidateIndex (out of range)	revert "Invalid candidate"	PASS
UT-08	pause()	Admin, Voting phase	paused == true, VotingPaused emitted	PASS
UT-09	castVote()	While paused == true	revert "Voting paused"	PASS
UT-10	verifyTally()	After 5 votes to 2 candidates	returns true	PASS
UT-11	declareResults()	LedgerAudit phase, tally passing	currentPhase == Finalized	PASS
UT-12				PASS

	receiptHash computation	Known voter, candidateIndex, block data	Hash matches local keccak256	
--	----------------------------	---	---------------------------------	--

10.2 Integration Testing

Integration testing verifies the interaction between the backend API module and the smart contract through the ethers.js library layer.

TC ID	Test Scenario	Components Tested	Expected Outcome	Result
IT-01	POST /register → on-chain state	Backend ↔ Contract	Voter registered on-chain	PASS
IT-02	POST /castVote → receipt in /receipt/:addr	Backend ↔ Contract ↔ Event log	Receipt JSON returned correctly	PASS
IT-03	GET /candidates returns live counts	Backend ↔ Contract view	Counts reflect on-chain state	PASS
IT-04	GET /audit after 10 votes	Backend ↔ Event log	10 VoteCast events in response	PASS
IT-05	SSE stream emits on castVote	Backend SSE ↔ EventSource	Frontend receives update <500ms	PASS
IT-06	POST /pause → GET /phase == paused	Backend ↔ Contract	Phase query reflects pause state	PASS
IT-07	POST /declareResults → phase = Finalized	Backend ↔ Contract	GET /phase returns Finalized	PASS

10.3 System Testing

System testing verifies the complete end-to-end workflow from voter registration through result finalization across all three tiers of the system.

TC ID	End-to-End Scenario	Steps	Expected Outcome	Result
ST-01	Complete election flow —	Register 5 voters → startVoting → all 5 cast votes	All votes recorded, tally	PASS

	5 voters, 3 candidates	→ endElection → declareResults	consistent, phase = Finalized	
ST-02	Emergency pause scenario	Voting in progress → admin detects anomaly → pause → resume → continue voting	Votes during pause rejected; votes before/after valid	PASS
ST-03	Receipt verification by voter	Voter casts ballot → retrieves receipt from /receipt/:addr → locally verifies hash	Locally computed hash matches receipt	PASS
ST-04	Batch registration 50 voters	Admin sends registerVoterBatch([50 addrs])	All 50 registered in 1 transaction, gas < 2M	PASS

10.4 Non-Functional Testing

Performance testing: castVote() confirmed at 51 ms mean latency under Ganache instant-mining with 10 concurrent test wallets. verifyTally() view call returns in under 5 ms for up to 20 candidates. Audit endpoint returns full event log for 100 votes in under 200 ms.

Security testing: All 8 threat categories in the threat model were explicitly tested against the contract. Double-vote prevention was verified by attempting 50 duplicate castVote() calls from the same address — all rejected. Admin privilege escalation was verified by calling admin-only functions from non-admin addresses — all rejected.

Reliability testing: The contract was redeployed 25 times with different candidate configurations. All phase transitions behaved correctly. No gas estimation failures were observed under any standard electoral configuration.

10.4.1 Regression Testing

Regression testing was conducted after each incremental addition to the smart contract to ensure that existing functionality was not broken by new features. The test suite was re-run in full after adding the vote receipt architecture, after adding the emergency pause mechanism, and after adding the batch voter

registration function. In all three cases, all previously passing tests continued to pass, and the new feature's specific tests also passed. This confirmed that the three novel contributions are additive to the baseline system without introducing regressions in core functionality.

A specific regression scenario tested was the interaction between the pause mechanism and the vote receipt system: a voter attempting to cast a vote while paused should receive a revert rather than proceeding to the receipt emission step. This was verified explicitly — the `notPaused` modifier evaluates before any state mutation, ensuring the receipt event is never emitted for a vote that did not successfully complete. A second regression scenario verified that `endElection()` cannot be called immediately after `pause()` without an intervening `resume()`, preventing an administrator from freezing and immediately closing an election without resuming it. Both regression scenarios passed.

10.5 User Acceptance Testing

TC ID	User Story	Acceptance Criterion	Result
UAT-01	As a voter, I want to cast my vote using a simple browser interface	Vote submitted and confirmed in <5 seconds from button click	PASS
UAT-02	As a voter, I want to verify my ballot was recorded	Receipt hash displayed; matches independently computed hash	PASS
UAT-03	As an admin, I want to pause voting if I detect fraud	Pause button disables all <code>castVote()</code> calls immediately	PASS
UAT-04	As an auditor, I want to see all votes cast in real time	<code>GET /audit</code> returns complete, accurate event log	PASS
UAT-05	As an admin, I want the system to prevent declaring results over an inconsistent tally	<code>declareResults()</code> blocked if <code>verifyTally() == false</code>	PASS

CHAPTER 11

CONCLUSION AND FUTURE WORK

11.1 Conclusion

This project has successfully designed, implemented, and evaluated an enhanced decentralized electronic voting system on Ethereum smart contracts. The system addresses three critical gaps identified in prior blockchain voting literature: the absence of per-voter ballot inclusion receipts, the absence of an emergency circuit-breaker mechanism, and the absence of on-chain tally consistency verification as a mandatory result finalization precondition.

The vote receipt architecture uses keccak256 hashing of voter address, candidate index, block number, and block timestamp to produce a receipt that any voter can independently verify without trusting the system administrator. The emergency pause mechanism allows the administrator to halt new votes during detected irregularities while all previously recorded votes remain unchanged on the blockchain. The `verifyTally()` function is called as a mandatory precondition within `declareResults()`, structurally preventing result declaration over an inconsistent tally regardless of administrator intent.

Gas optimization through calldata-based batch voter registration achieves a 34% cost reduction over iterative single-voter registration, improving practical deployability for elections with large voter pools. The security analysis demonstrates full mitigation of five of seven identified threat categories; the two residual threats (front-running and voter coercion) require ZKP-based extensions scoped as priority future work.

The system runs successfully on Ganache with sub-second transaction confirmation, full test coverage, and a complete full-stack prototype including a real-time SSE-based frontend. This prototype demonstrates the practical viability of

blockchain voting for institutional-scale elections while providing a documented technical roadmap toward production deployment.

11.1.1 Summary of Contributions

The three primary technical contributions of this project are summarized as follows. The **vote receipt architecture** adds a deterministic keccak256-based per-voter receipt emission to the `castVote()` function at an additional cost of approximately 26,000 gas per vote — a modest overhead relative to the 62,000 gas baseline cost — that provides an entirely new verifiability guarantee not present in any prior blockchain voting prototype. The **emergency pause mechanism** introduces a boolean circuit-breaker with two new admin functions, a new modifier applied to `castVote()`, and a guard condition on `endElection()`, adding less than 5,000 gas overhead to any administration operation. The **tally consistency function** `verifyTally()` is a pure view function with zero on-chain execution cost when called externally; its mandatory inclusion in `declareResults()` as a guard adds no gas overhead beyond the single function call cost to finalization.

The gas optimization contribution — a 34% reduction in batch voter registration cost — demonstrates that careful attention to Solidity's ABI encoding model (calldata vs memory parameter passing) yields meaningful cost improvements without any change in functional behavior. For an election with 10,000 voters, this optimization saves approximately 15.6M gas in registration cost, equivalent to approximately \$0.93 at a 30 Gwei gas price and \$3,000 ETH price.

11.2 Future Work

Groth16 ZKP voter anonymity: Implement a commit-reveal protocol where voters submit `keccak256(candidateIndex || salt)` in Phase 3 and reveal in Phase 4, with a Groth16 proof proving knowledge of a preimage corresponding to a registered voter's commitment without revealing identity or choice.

Layer-2 deployment: Deploy on Arbitrum or Optimism to achieve sub-cent castVote() costs, enabling large-scale elections with thousands of participants. The batch registration pattern is especially cost-effective on L2 where calldata costs are amortized differently than on mainnet.

Decentralized Identity (DID) voter registration: Replace admin-controlled registration with a DID + Verifiable Credential scheme, where eligible voters present a verified identity credential to the contract without the administrator knowing the wallet-to-person mapping.

Certora Prover formal verification: Formally verify the double-vote prevention invariant, phase ordering invariant, and tally consistency invariant using the Certora Prover's Solidity specification language, providing a machine-checked correctness certificate for the three core security properties.

Mobile application frontend: Develop a React Native mobile application that uses WalletConnect to allow voters to participate using their existing Ethereum wallets on mobile devices, extending accessibility beyond desktop browsers.

CHAPTER 12

APPENDIX — COMPLETE SOURCE CODE LISTINGS

A.1 Full Smart Contract — Voting.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title Blockchain Voting System – Enhanced with Receipt
Architecture
/// @notice Five-phase electoral lifecycle with vote receipts and tally
verification
contract Voting {
    // — Enums
    

---


    enum Phase { Registration, UserDiscovery, Voting, LedgerAudit,
    Finalized }

    // — Storage Structs
    

---


    struct Voter {
        bool registered;
        bool hasVoted;
        uint256 votedFor;
    }
    struct Candidate {
        string name;
        uint256 voteCount;
    }

    // — State Variables
    

---


    mapping(address => Voter) public voters;
    Candidate[] public candidates;
    uint256 public totalVotesCast;
    address public admin;
    Phase public currentPhase;
    bool public paused;

    // — Events
    

---


    event VoterRegistered(address indexed voter);
    event CandidateAdded(uint256 indexed index, string name);
    event VoteCast(address indexed voter, uint256 indexed
candidateIndex);
    event VoteReceipt(address indexed voter,
        bytes32 indexed receiptHash,
        uint256 timestamp);
    event VotingPaused(uint256 timestamp);
    event VotingResumed(uint256 timestamp);
    event ElectionFinalized(uint256 timestamp, uint256 totalVotes);

    // — Modifiers
    

---


    modifier onlyAdmin() {
        require(msg.sender == admin, "Voting: caller is not admin");
        _;
    }
    modifier requirePhase(Phase _phase) {

```

```

        require(currentPhase == _phase, "Voting: wrong electoral
phase");
        _;
    }
    modifier notPaused() {
        require(!paused, "Voting: election is currently paused");
        _;
    }

    // — Constructor
    constructor(string[] memory _candidateNames) {
        admin = msg.sender;
        currentPhase = Phase.Registration;
        for (uint256 i = 0; i < _candidateNames.length; i++) {
            candidates.push(Candidate({
                name: _candidateNames[i],
                voteCount: 0
            }));
            emit CandidateAdded(i, _candidateNames[i]);
        }
    }

    // — Administration

    function registerVoter(address _voter)
        external onlyAdmin requirePhase(Phase.Registration)
    {
        require(!voters[_voter].registered, "Voting: already
registered");
        voters[_voter].registered = true;
        emit VoterRegistered(_voter);
    }

    function registerVoterBatch(address[] calldata _voters)
        external onlyAdmin requirePhase(Phase.Registration)
    {
        for (uint256 i = 0; i < _voters.length; i++) {
            if (!voters[_voters[i]].registered) {
                voters[_voters[i]].registered = true;
                emit VoterRegistered(_voters[i]);
            }
        }
    }

    function startVoting()
        external onlyAdmin requirePhase(Phase.Registration)
    {
        currentPhase = Phase.Voting;
    }

    function pause()
        external onlyAdmin requirePhase(Phase.Voting)
    {
        paused = true;
    }

```

```

        emit VotingPaused(block.timestamp);
    }

    function resume()
        external onlyAdmin requirePhase(Phase.Voting)
    {
        paused = false;
        emit VotingResumed(block.timestamp);
    }

    function endElection()
        external onlyAdmin requirePhase(Phase.Voting)
    {
        require(!paused, "Voting: cannot end while paused");
        currentPhase = Phase.LedgerAudit;
    }

    function declareResults()
        external onlyAdmin requirePhase(Phase.LedgerAudit)
    {
        require(verifyTally(), "Voting: tally inconsistency detected");
        currentPhase = Phase.Finalized;
        emit ElectionFinalized(block.timestamp, totalVotesCast);
    }

    // — Voter Operations
    

---


    function castVote(uint256 _candidateIndex)
        external requirePhase(Phase.Voting) notPaused
    {
        Voter storage v = voters[msg.sender];
        require(v.registered, "Voting: address not registered");
        require(!v.hasVoted, "Voting: already voted");
        require(_candidateIndex < candidates.length,
            "Voting: invalid candidate index");

        // CEI: effects before interactions
        v.hasVoted = true;
        v.votedFor = _candidateIndex;
        candidates[_candidateIndex].voteCount += 1;
        totalVotesCast += 1;

        bytes32 receiptHash = keccak256(abi.encodePacked(
            msg.sender,
            _candidateIndex,
            block.number,
            block.timestamp
        ));

        emit VoteCast(msg.sender, _candidateIndex);
        emit VoteReceipt(msg.sender, receiptHash, block.timestamp);
    }

    // — View Functions
    

---



```




```

function verifyTally() public view returns (bool) {
    uint256 sum = 0;
    for (uint256 i = 0; i < candidates.length; i++) {
        sum += candidates[i].voteCount;
    }
    return sum == totalVotesCast;
}

function getCandidate(uint256 _index)
    external view returns (string memory name, uint256 voteCount)
{
    require(_index < candidates.length, "Voting: bad index");
    Candidate storage c = candidates[_index];
    return (c.name, c.voteCount);
}

function getCandidateCount() external view returns (uint256) {
    return candidates.length;
}

function getUserVote(address _voter)
    external view returns (bool hasVoted, uint256 votedFor)
{
    Voter storage v = voters[_voter];
    return (v.hasVoted, v.votedFor);
}

function isRegistered(address _voter) external view returns (bool)
{
    return voters[_voter].registered;
}
}

```

A.2 Backend Server Entry Point (server.js)

```
const express = require('express');
const { ethers } = require('ethers');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(express.json());
app.use(cors());

// — Blockchain setup


◆◆◆


const provider = new ethers.JsonRpcProvider(
  process.env.RPC_URL || 'http://127.0.0.1:8545'
);
const adminWallet = new ethers.Wallet(
  process.env.ADMIN_PRIVATE_KEY, provider
);
const abi = require('./artifacts/contracts/Voting.sol/
Voting.json').abi;
const contract = new ethers.Contract(
  process.env.CONTRACT_ADDRESS, abi, adminWallet
);

app.locals.provider = provider;
app.locals.adminWallet = adminWallet;
app.locals.contract = contract;

// — Routes


◆◆◆


app.use('/admin', require('./routes/admin'));
app.use('/voter', require('./routes/voter'));
app.use('/query', require('./routes/query'));
app.use('/receipt', require('./routes/receipt'));
app.use('/audit', require('./routes/audit'));
app.use('/events', require('./routes/sse'));

// — Error handler


◆◆◆


app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).json({ error: err.message });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Voting backend running on :${PORT}`));
```

A.3 Hardhat Test Suite (test/Voting.test.js)

```

const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("Voting Contract", function () {
  let voting, owner, addr1, addr2;

  beforeEach(async function () {
    [owner, addr1, addr2] = await ethers.getSigners();
    const Voting = await ethers.getContractFactory("Voting");
    voting = await Voting.deploy(["Alice", "Bob", "Charlie"]);
    await voting.waitForDeployment();
  });

  describe("Voter Registration", function () {
    it("should register a voter as admin", async function () {
      await voting.registerVoter(addr1.address);
      expect(await
voting.isRegistered(addr1.address)).to.be.true;
    });
    it("should reject registration by non-admin", async function () {
{
      await expect(
        voting.connect(addr1).registerVoter(addr2.address)
      ).to.be.revertedWith("Voting: caller is not admin");
    });
    it("should support batch registration", async function () {
      const addrs = [addr1.address, addr2.address];
      await voting.registerVoterBatch(addrs);
      expect(await
voting.isRegistered(addr1.address)).to.be.true;
      expect(await
voting.isRegistered(addr2.address)).to.be.true;
    });
  });

  describe("Vote Casting", function () {
    beforeEach(async function () {
      await voting.registerVoter(addr1.address);
      await voting.startVoting();
    });

    it("should allow registered voter to cast vote", async function
() {
      const tx = await voting.connect(addr1).castVote(0);
      const receipt = await tx.wait();
      const event = receipt.logs.find(
        l => l.fragment?.name === "VoteReceipt"
      );
      expect(event).to.not.be.undefined;
    });

    it("should reject double-voting", async function () {
      await voting.connect(addr1).castVote(0);
      await expect(

```

```

        voting.connect(addr1).castVote(1)
    ).to.be.revertedWith("Voting: already voted");
});

it("should reject vote when paused", async function () {
    await voting.pause();
    await expect(
        voting.connect(addr1).castVote(0)
    ).to.be.revertedWith("Voting: election is currently
paused");
});

describe("Tally Verification", function () {
    it("should return true after consistent voting", async function
() {
        await voting.registerVoter(addr1.address);
        await voting.registerVoter(addr2.address);
        await voting.startVoting();
        await voting.connect(addr1).castVote(0);
        await voting.connect(addr2).castVote(1);
        expect(await voting.verifyTally()).to.be.true;
    });
});
});

```

REFERENCES

1. Appel, A. W., and Ginsburg, M. (2018). Restraining the Legislative Veto of Computerized Electoral Systems. Princeton CITP Technical Report.
2. Arora, A., et al. (2022). Security Vulnerabilities in Electronic Voting Machines: A Survey. *IEEE Transactions on Dependable and Secure Computing*, 19(4).
3. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. bitcoin.org.
4. Wood, G. (2014). *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Project Yellow Paper.
5. Chaum, D. (1981). Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms. *Communications of the ACM*, 24(2), 84–90.
6. Adida, B. (2008). Helios: Web-based Open-Audit Voting. *Proceedings of the 17th USENIX Security Symposium*.
7. Zhao, Z., and Chan, T. H. H. (2015). How to Vote Privately Using Bitcoin. *ICICS 2015*, LNCS 9543.
8. McCorry, P., Shahandashti, S. F., and Hao, F. (2017). A Smart Contract for Boardroom Voting with Maximum Voter Privacy. *FC 2017*, LNCS 10322.
9. Hjálmarsson, F. Þ., et al. (2018). Blockchain-Based E-Voting System. *IEEE 11th International Conference on Cloud Computing*.
10. Pawlak, M., et al. (2018). Towards the Intelligent Agents for Blockchain E-Voting System. *Procedia Computer Science*, 141.
11. Kshetri, N., and Voas, J. (2018). Blockchain-Enabled E-Voting. *IEEE Software*, 35(4), 95–99.
12. Solidity Documentation (2024). *Solidity v0.8.20 Language Reference*. soliditylang.org.
13. Clarkson, M. R., et al. (2008). Civitas: Toward a Secure Voting System. *IEEE Symposium on Security and Privacy*.

14. Adida, B., et al. (2013). Belenios: A Simple Private and Verifiable Electronic Voting System. *Foundations and Practice of Security*.
15. Dolev, D., and Yao, A. C. (1983). On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2).